

Lab 18: API Integration: Connecting to External Services with Error Handling

Task 1: Movie Database API Integration

Prompt:

1. Create Python program to fetch movie details.
2. Take movie name as input.
3. Use free API without API key.
4. Show title, year, rating and plot.
5. Handle errors properly.

Code:

```
import requests

movie = input("Enter movie name: ")

url = f"https://www.omdbapi.com/?t={movie}&apikey=thetvdb" # free demo key

try:
    response = requests.get(url, timeout=5)
    data = response.json()

    if data["Response"] == "False":
        print("Error: Movie not found")
    else:
        print("\n--- Movie Details ---")
        print("Title:", data.get("Title"))
        print("Year:", data.get("Year"))
        print("Rating:", data.get("imdbRating"))
        print("Plot:", data.get("Plot"))

except requests.exceptions.Timeout:
    print("Error: Timeout")

except requests.exceptions.RequestException:
    print("Error: Network problem")

except:
    print("Error occurred")
```

Output:

```
... Enter movie name: Titanic

--- Movie Details ---
Title: Titanic
Year: 1997
Rating: 8.0
Plot: A seventeen-year-old aristocrat falls in love with a kind but poor artist aboard the luxurious, ill-fated R.M.S. Titanic.
```

Explanation:

1. Uses free OMDb demo API key.
2. Fetches movie details using requests.
3. Displays important fields clearly.
4. Handles errors safely.

Task 2: Transport API (Using Public Transport API)

Prompt:

1. Create program to get transport arrival times.
2. Take stop ID from user.
3. Show next 3 arrivals.
4. Retry if request fails.
5. Handle errors properly.

Code:

```
import requests
import time

stop_id = input("Enter stop ID (example: 490008660N): ")

url = f"https://api.tfl.gov.uk/StopPoint/{stop_id}/Arrivals"

for attempt in range(2):
    try:
        response = requests.get(url, timeout=5)

        if response.status_code != 200:
            print("Server error")
            break

        data = response.json()

        if not data:
            print("Invalid stop ID")
            break

        print("\nNext arrivals:")
        for i in range(min(3, len(data))):
            print(data[i]["lineName"], "-", data[i]["destinationName"])

        break

    except requests.exceptions.Timeout:
        print("Timeout... retrying")
```

Output:

... Enter stop ID (example: 490008660N): 490008660N

Next arrivals:

214 - Highgate Village

214 - Highgate Village

214 - Highgate Village

Explanation:

1. Uses London transport free API.
2. Fetches arrival data using stop ID.
3. Shows next 3 arrivals.
4. Retry works if timeout occurs.

Task 3: Stock Market API

Prompt :

1. Create program to fetch stock data.
2. Take stock symbol as input.
3. Show price, high and low.
4. Use free API.
5. Handle invalid symbol and errors.

Code:

```
import requests

symbol = input("Enter stock symbol (example: AAPL): ")

url = f"https://query1.finance.yahoo.com/v7/finance/quote?symbols={symbol}"

try:
    response = requests.get(url, timeout=5)
    data = response.json()

    result = data["quoteResponse"]["result"]

    if not result:
        print("Invalid stock symbol")
    else:
        stock = result[0]
        print("\n--- Stock Details ---")
        print("Price:", stock.get("regularMarketPrice"))
        print("High:", stock.get("regularMarketDayHigh"))
        print("Low:", stock.get("regularMarketDayLow"))

except requests.exceptions.Timeout:
    print("Timeout error")

except:
    print("Error occurred")
```

Output:

```
... Enter stop ID (example: 490008660N): 490008660N

Next arrivals:
214 - Highgate Village
214 - Highgate Village
214 - Highgate Village
```

Explanation:

1. Uses Yahoo Finance free API.
2. Fetches real-time stock data.
3. Extracts price, high and low.
4. Handles invalid symbols safely.

Task 4: Geolocation API (Working)

Prompt:

1. Create program to find location using IP.
2. Take IP address from user.
3. Validate input format.
4. Show country, city and ISP.
5. Handle API and errors.

Code:

```
import requests
import re

ip = input("Enter IP address: ")

pattern = r"^\d{1,3}(\.\d{1,3}){3}$"

if not re.match(pattern, ip):
    print("Invalid IP format")
else:
    url = f"http://ip-api.com/json/{ip}"

    try:
        response = requests.get(url, timeout=5)
        data = response.json()

        if data["status"] != "success":
            print("API failed")
        else:
            print("\n--- Location ---")
            print("Country:", data.get("country"))
            print("City:", data.get("city"))
            print("ISP:", data.get("isp"))

    except:
        print("Error occurred")
```

Output:

```
... Enter IP address: 8.8.8.8

--- Location ---
Country: United States
City: Ashburn
ISP: Google LLC
```

Explanation:

1. Validates IP format using regex.
2. Uses free geolocation API.
3. Displays location details.
4. Handles errors properly.

Task 5: Payment Simulation

Prompt:

1. Create program to simulate payment API.
2. Take amount, currency and method.
3. Send POST request.
4. Handle HTTP errors.
5. Save failed transactions.

Code:

```
import requests

amount = input("Enter amount: ")
currency = input("Enter currency: ")
method = input("Enter payment method: ")

url = "https://httpbin.org/post"

data = {
    "amount": amount,
    "currency": currency,
    "method": method
}

try:
    response = requests.post(url, json=data, timeout=5)

    if response.status_code == 200:
        print("\nPayment Successful")
        print("Response ID:", response.json().get("headers"))

    else:
        print("Payment Failed")

        with open("failed.txt", "a") as f:
            f.write(str(data) + "\n")

except requests.exceptions.RequestException:
    print("Network error")
```

Output:

```
... Enter amount: 100
    Enter currency: INR
    Enter payment method: card

Payment Successful
Response ID: {'Accept': '*/*', 'Accept-Encoding': 'gzip, deflate, br, zstd',
```

Explanation:

1. Uses httpbin API to simulate payment.
2. Sends POST request with user data.
3. Shows success or failure message.
4. Failed data is saved in file.